

TITLE: Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

Title of the project :

Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

S.no	Name	College	Contact	Register No
1	RAGHUL K	Sri Krishna Adithya College of Arts and Science	23bsct142raghulk@skacas.ac.in	2326K0268
2	DHARSHINI M	Sri Krishna Adithya College of Arts and Science	23bsct112dharshinim@skacas.ac.in	2326K0238
3	SANJANA S	Sri Krishna Adithya College of Arts and Science	23bsct153sanjanas@skacas.ac.in	2326K0279
4	SANJAY K	Sri Krishna Adithya College of Arts and Science	23bsct154sanjayk@skacas.ac.in	2326K0280

INTRODUCTION:

As the digital environment continues to evolve, organizations are increasingly exposed to sophisticated cyber threats. This growing risk highlights the critical need for rapid and efficient incident response mechanisms to protect sensitive information, secure digital infrastructure, and maintain stakeholder confidence. To address these challenges, this project proposes the development of an AI-powered incident response platform designed to support cybersecurity teams by automating incident triage and improving the efficiency of response activities.

With the rise of cloud computing and remote work, the traditional network perimeter has become less defined, making legacy VPN-based remote access models less effective. These conventional systems often provide broad network-level access, which increases the attack surface and allows potential attackers to move laterally within the network. To overcome these limitations, this project adopts a Zero Trust security architecture, built on the principle of “never trust, always verify.” Instead of relying on network location to determine trust, access decisions are made based on identity, device posture, and contextual factors.

A key component of this approach is the implementation of Zscaler Private Access (ZPA), a cloud-based Zero Trust Access solution that enables secure, application-level connectivity. Unlike traditional VPNs that place users inside the corporate network, ZPA creates secure, direct connections between authenticated users and the specific applications they are authorized to use. This service-initiated architecture hides internal networks and IP addresses from the internet, ensuring that applications remain invisible to unauthorized users while protecting them from threats such as port scanning and distributed denial-of-service (DDoS) attacks.

The solution also enforces a strict least-privilege access model, ensuring that users and devices receive only the minimum permissions required to perform their roles. By separating application access from network access, the system prevents users from viewing or interacting with other network resources, thereby eliminating opportunities for lateral movement within the environment. This strategy significantly strengthens the organization’s security posture, reduces the overall attack surface, and simplifies access management while providing a more seamless experience for users.

ABSTRACT

The rapid expansion of hybrid work environments and cloud services has reduced the effectiveness of traditional perimeter-based security systems such as Virtual Private Networks (VPNs). These legacy approaches typically grant users broad access to internal networks, which increases security risks such as lateral threat movement and a larger attack surface. This project addresses these challenges by implementing a Zero Trust Architecture (ZTA) using Zscaler Private Access (ZPA). The main goal is to replace the traditional network-based access model with an identity-based approach that enforces the principle of least privilege.

The proposed solution deploys ZPA's cloud-based platform to create secure, application-level connections through identity-aware micro-tunnels. Unlike conventional VPNs that connect users directly to the internal network, this system establishes secure one-to-one connections between authenticated users and specific authorized applications. As a result, the internal network and other unauthorized resources remain hidden from users, significantly reducing opportunities for reconnaissance and cyberattacks. Access policies are dynamically applied based on user identity, device security posture, and contextual factors, ensuring that users can only access the resources necessary for their job roles.

By implementing ZPA, the project effectively separates application access from overall network access, resulting in a reduced attack surface and preventing lateral movement within the network. The solution provides a secure, scalable, and efficient remote access framework that aligns with modern cybersecurity best practices. Adopting this identity-centered, least-privilege access model improves the organization's security posture while also simplifying management and enhancing the user experience.

Use Case Scenarios

Scenario 1: Finance Department Access to ERP System

A senior accountant from the finance team needs remote access to the organization's Enterprise Resource Planning (ERP) system to complete monthly payroll activities. The employee connects using a company-issued laptop from an external location. After launching the ZPA Client Connector, the user authenticates through Microsoft Azure Active Directory using

multi-factor authentication. The Zero Trust policy engine evaluates multiple factors such as the user's identity, membership in the Finance_ERP_Users group, device security status (active antivirus and enabled disk encryption), and connection context. Once verification is successful, Zscaler Private Access establishes a secure micro-tunnel directly to the ERP application. The user is able to perform required payroll tasks while remaining completely isolated from other internal systems such as development environments or HR databases.

Output

- Successful authentication based on identity and group membership
 - Secure micro-tunnel created only for the ERP application
 - Other internal resources remain hidden and inaccessible
-

Scenario 2: Contractor Access to a Project Repository

An external consultant is hired for a temporary software development project and needs access to a specific internal Git repository. The contractor is registered as a guest user in the identity management system and placed in the "Project_X_Contractors" group. When connecting through the ZPA client, the system verifies the user's credentials and evaluates the device's security posture, including operating system patch status and firewall configuration. Based on predefined policies in Zscaler Private Access, the contractor is allowed to access only the designated Git repository via SSH (port 22) or HTTPS (port 443) during working hours. The consultant can perform repository operations such as cloning and pushing code but is unable to access other internal services like file servers or the corporate intranet.

Output

- Access limited by role and working hours
 - Contractor restricted to a single application resource
 - Lateral movement across internal systems prevented
-

Scenario 3: Executive Access to Confidential Board Documents

A company executive needs to review confidential board meeting documents stored on an internal file server while traveling abroad. The executive connects using a tablet device. The system integrates with the organization's mobile device management platform to verify that the device meets compliance requirements, including encryption, passcode protection, and absence of jailbreak modifications. After successful authentication, the Zero Trust policy evaluates conditions such as membership in the "Executive_Board" group, device compliance status, and connection from an approved geographic region. If these conditions are satisfied, Zscaler Private Access provides a secure connection to the "Board_Documents" file share. Data transmission occurs through encrypted TLS channels, while attempts to access other network resources are blocked and recorded for auditing.

Output

- Context-aware authorization based on identity, device compliance, and location
 - Controlled access to the confidential document repository
 - Detailed activity logging for compliance and auditing
-

Scenario 4: IT Administrator Access to Server Management Interfaces

A system administrator must access server management interfaces to perform routine system maintenance. Instead of using a traditional VPN that exposes the entire management network, the organization configures Zscaler Private Access to treat each server management interface as an independent application segment. The administrator authenticates and is presented only with servers that match their assigned role, such as Linux servers for the "Linux_Admins" group or Windows servers for the "Windows_Admins" group. When the administrator connects to a Linux server via SSH or a Windows server through RDP, ZPA establishes a secure micro-tunnel to the exact server and port required. This prevents the administrator from scanning the network or accessing unauthorized systems.

Output

- Fine-grained micro-segmentation at the server level
 - Role-based access to specific administrative interfaces
 - Network scanning and reconnaissance capabilities eliminated
-

Scenario 5: Mergers and Acquisitions Integration

After acquiring a smaller company, the parent organization must temporarily provide the new HR team access to its internal employee benefits enrollment application. Users from the acquired company are added as external collaborators in the identity provider and assigned to the “Acquired_HR” group. Access policies in Zscaler Private Access allow these users to connect only to the internal Benefits_App web application. Once authenticated through the ZPA client, users are granted direct application-level access without exposure to the broader corporate network. They cannot scan, ping, or interact with other internal systems. After the six-month transition period, the temporary access policy is removed, completing the integration process securely.

Output

- Temporary application-specific access for external collaborators
 - Internal network remains hidden from external users
 - Access easily revoked through policy updates rather than infrastructure changes
-

Stage – 1

Title:

Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

Overview

As cyber threats continue to grow in complexity and frequency, organizations must adopt more advanced security strategies to protect their digital infrastructure. At the same time, the traditional network perimeter has become less relevant due to the rise of cloud services, mobile devices, and remote work. Conventional VPN-based remote access solutions often provide users with broad network access, which increases the attack surface and allows potential attackers to move laterally within the system once access is gained. To address these challenges, this project focuses on implementing a Zero Trust security architecture, which operates on the principle of “never trust, always verify.” Rather than relying on network location as a measure of trust, access decisions are based on verified identity, device security status, and contextual factors.

A key component of this project is the deployment of Zscaler Private Access, a cloud-delivered Zero Trust Access solution that enables secure, application-level connectivity. Unlike traditional VPN solutions that connect users directly to the internal network, ZPA creates secure and direct connections between authenticated users and the specific applications they are authorized to access. This service-initiated architecture ensures that internal networks and IP addresses remain hidden from the public internet. As a result, applications become invisible to unauthorized users, effectively reducing exposure to threats such as port scanning and distributed denial-of-service attacks.

The project also implements a least-privilege access model, ensuring that users and devices receive only the permissions required to perform their assigned tasks. By separating application access from network-level access, the solution prevents users from discovering or interacting with other internal resources. This approach significantly limits opportunities for lateral movement within the network. Overall, the adoption of Zero Trust access through ZPA

strengthens the organization's security posture, reduces the overall attack surface, improves the user experience, and minimizes the operational burden associated with maintaining traditional remote access infrastructure.

List of Vulnerability

S. No.	Vulnerability Name	CWE No.
1.	Improper Limitation of a Pathname to a Restricted Directory	22
2.	Unimplemented or Unsupported Feature in UI	447
3.	Weak Encoding for Password	261
4.	Improper Input Validation	20
5.	External Control of File Name or Path	73
6.	Empty Password in Configuration File	258
7.	Insufficient verification of data Authenticity	345
8.	Improper output Neutralization for Logs	117
9.	Generation of error messages Containing sensitive information	209

1. Vulnerability name:

Improper Limitation of a
pathname to a restricted
directory

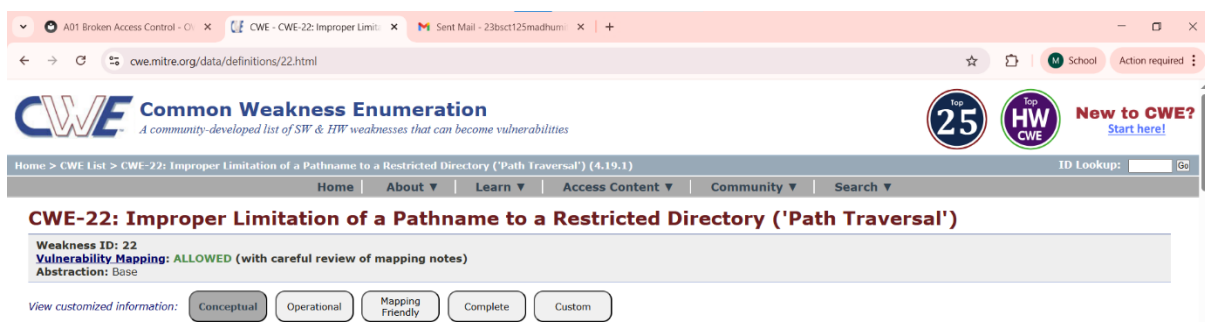
CWE: 22

Description : CWE-22 Improper Limitation of a Pathname to a Restricted Directory, commonly called Path Traversal, is a security vulnerability where an application fails to properly restrict user-supplied file paths, allowing attackers to access files or directories outside the intended directory. By manipulating input with sequences like ../, an attacker can traverse the file system and potentially read sensitive files (such as configuration files, system credentials, or source code) or access restricted resources on the server. This vulnerability occurs when applications do not properly validate, sanitize, or restrict file path inputs before using them in file system operations.

Business impact :

Path Traversal vulnerabilities allow attackers to access files and directories outside the intended application directory by manipulating file path inputs. From a business perspective, this vulnerability can cause significant damage. Attackers may gain unauthorized access to sensitive files such as configuration files, credentials, customer data, or internal documents. This can lead to data breaches, exposing confidential information and violating privacy regulations. Such incidents often result in financial losses due to regulatory fines, legal actions, incident response costs, and potential compensation to affected customers. In addition, exploitation of this vulnerability can disrupt normal operations if critical system files are modified or deleted, leading to service downtime and reduced productivity.

Vulnerability Path : <https://cwe.mitre.org/data/definitions/22.html>



2.Vulnerability Name : Unimplemented or Unsupported Feature in UI

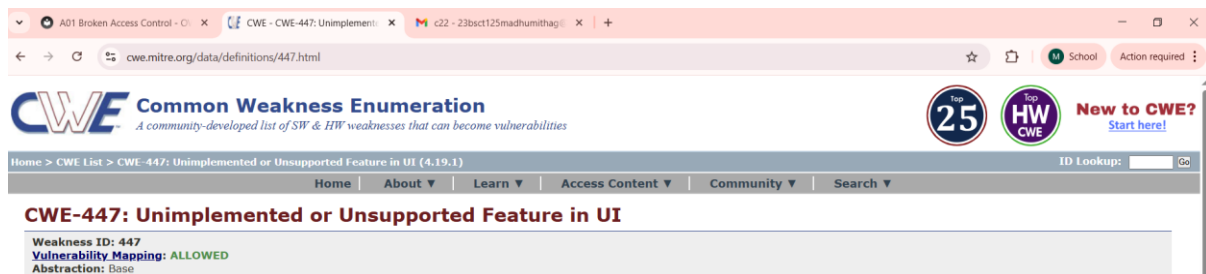
CWE : 447

Description : Use of Obsolete Function refers to a security weakness that occurs when a program uses outdated or deprecated functions that are no longer considered safe or recommended. These obsolete functions may contain known vulnerabilities, lack modern security protections, or behave in ways that can lead to security risks such as buffer overflows, improper memory handling, or data exposure.

Business Impact : When a user interface exposes features that are not actually implemented or supported, it can create significant business risks. First, it leads to loss of user trust and credibility, as customers expect advertised features to function properly. When they encounter non-working options, they may perceive the product as unreliable or poorly maintained. Second, it can result in increased support and maintenance costs. Users often report

such issues to customer support, increasing workload and operational expenses. Development teams may also need emergency fixes or UI redesigns to correct the misleading functionality.

Vulnerability Path : <https://cwe.mitre.org/data/definitions/447.html>



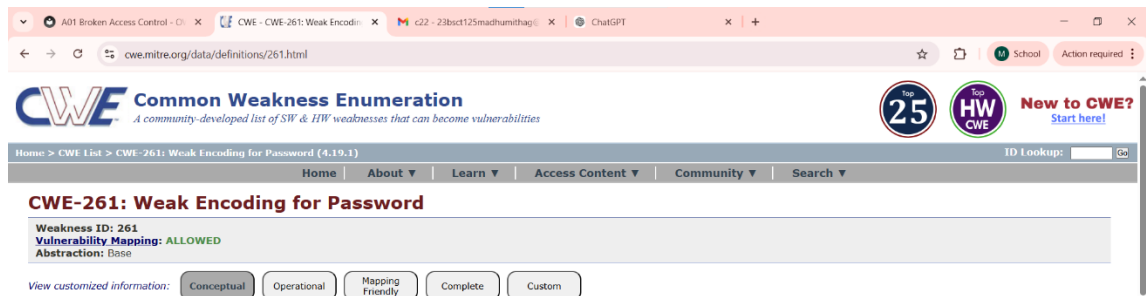
3. Vulnerability Name : Weak Encoding for Password

CWE : 261

Description : Weak Encoding for Password is a security weakness that occurs when an application stores or transmits passwords using simple or reversible encoding methods instead of strong cryptographic hashing. Encoding techniques such as Base64 or other basic transformations can easily be decoded by attackers, allowing them to recover the original passwords.

Business impact : CWE-261 Weak Encoding for Password refers to storing or transmitting passwords using weak or reversible encoding methods (such as Base64) instead of strong cryptographic hashing. This vulnerability can have significant business consequences. If attackers gain access to encoded passwords, they can easily decode them and obtain valid user credentials. This may lead to unauthorized system access, data breaches, and account takeovers. For organizations, such incidents can result in financial losses due to fraud, operational disruption, and incident response costs. Additionally, compromised credentials may allow attackers to move laterally within internal systems, exposing sensitive corporate or customer data.

Vulnerability Path : <https://cwe.mitre.org/data/definitions/261.html>



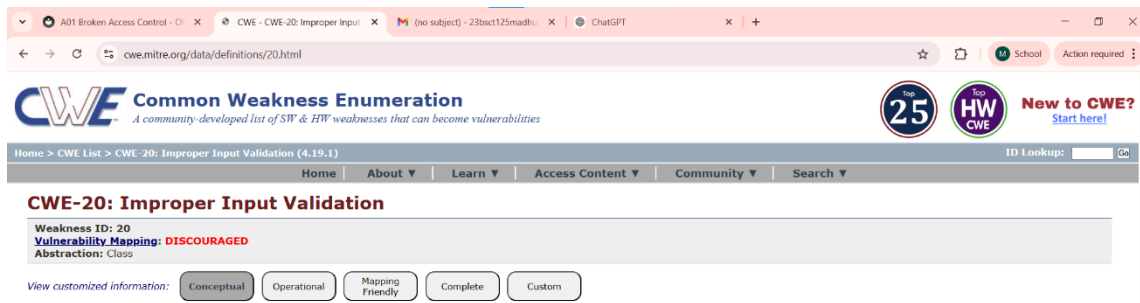
4. Vulnerability Name : Improper Input Validation

CWE : 20

Description : Improper Input Validation is a security vulnerability that occurs when a program fails to properly check or validate input provided by users or external sources. Without proper validation, attackers may send malicious or unexpected data that can cause the system to behave incorrectly, leading to issues such as injection attacks, system crashes, or unauthorized access.

Business impact : CWE-20 Improper Input Validation occurs when an application fails to properly validate user inputs before processing them. This weakness can allow attackers to submit malicious or unexpected data that the system is not designed to handle. From a business perspective, this vulnerability can lead to serious security and operational risks. Attackers may exploit improper input validation to perform attacks such as injection, unauthorized data access, or system manipulation. As a result, sensitive information like customer records, financial data, or internal business details may be exposed or altered. The financial impact can be significant. Organizations may incur costs related to incident response, system recovery, legal actions, and regulatory penalties if sensitive data is compromised. In industries that must comply with data protection regulations, failure to properly validate inputs may be considered negligence, resulting in compliance violations and fines.

Vulnerability Path : <https://cwe.mitre.org/data/definitions/20.html>



5. Vulnerability name : External Control of File Name or Path

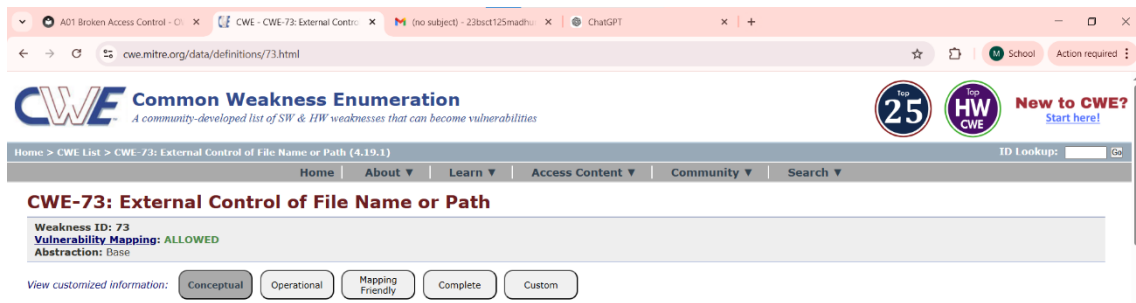
CWE : 73

Description : External Control of File Name or Path is a security vulnerability that occurs when an application allows external input, such as user-supplied data, to determine the file name or file path used in file operations. If this input is not properly validated or restricted, attackers may manipulate the path to access unauthorized files, overwrite important system files, or execute malicious actions (for example through path traversal attacks).

Business impact : CWE-73 External Control of File Name or Path occurs when an application allows external users to control or influence file names or file paths without proper validation or restriction. This vulnerability can allow attackers to manipulate file operations, potentially accessing or modifying files that should not be exposed. From a business perspective, this weakness can lead to serious security, financial, and operational consequences.

Attackers may exploit this issue to perform path traversal attacks, enabling them to read sensitive files such as configuration files, system credentials, or internal documents. Exposure of such information can result in data breaches involving confidential customer data, intellectual property, or business secrets.

Vulnerability Path : <https://cwe.mitre.org/data/definitions/73.html>



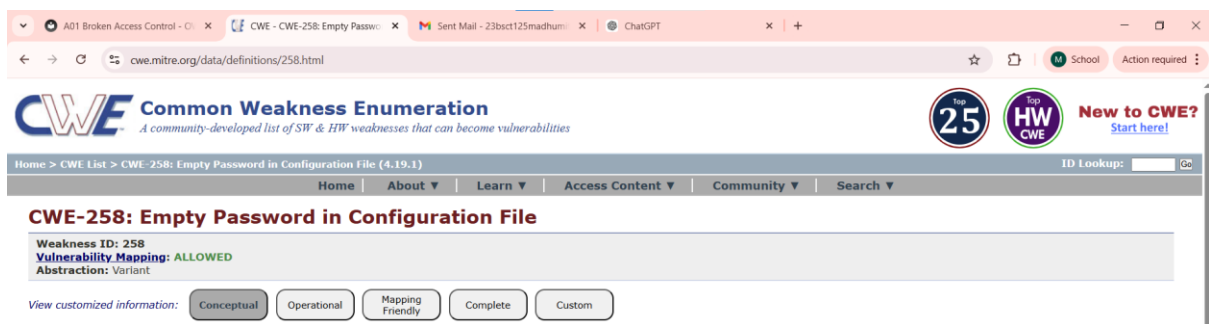
6. Vulnerability name : Empty Password in Configuration File

CWE :258

Description : Empty Password in Configuration File is a security vulnerability that occurs when a configuration file contains an account or service with an empty password. This means the system allows authentication without requiring any password, making it extremely easy for attackers to gain unauthorized access. If an attacker discovers such a configuration, they can log in or access sensitive services without any authentication barrier. To prevent this issue, all accounts and services should be configured with strong, non-empty passwords and secure authentication mechanisms.

Business Impact : CWE-258 Empty Password in Configuration File occurs when a configuration file contains an account or service with a blank or empty password. This security weakness significantly increases the risk of unauthorized access because attackers can authenticate to systems, databases, or services without needing valid credentials. From a business perspective, this vulnerability can lead to serious security and operational risks. If exploited, attackers may gain direct access to sensitive systems, allowing them to view, modify, or delete critical business data.

Vulnerability Path : <https://cwe.mitre.org/data/definitions/258.html>



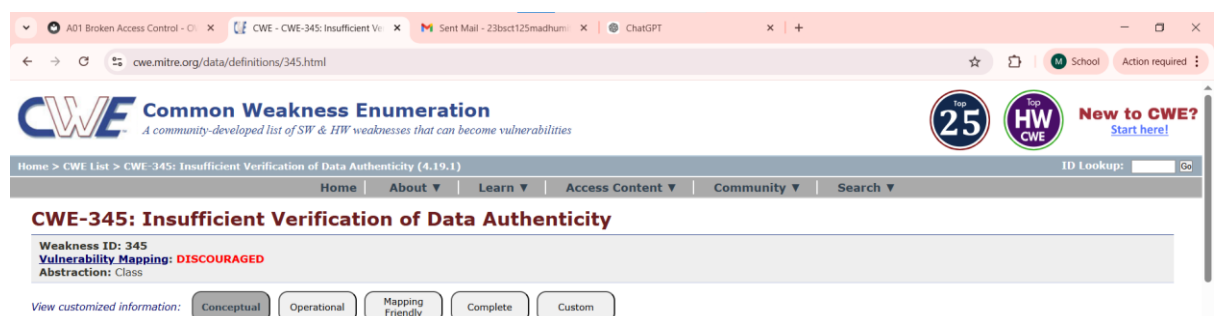
7.Vulnerability name :Insufficient Verification of Data Authenticity

CWE :345

Description : Insufficient Verification of Data Authenticity is a security vulnerability that occurs when a system does not properly verify whether received data is genuine and comes from a trusted source. Without adequate authenticity checks, attackers may modify, forge, or inject malicious data into the system, potentially leading to unauthorized actions, incorrect processing, or security breaches. This weakness often arises when applications fail to use mechanisms such as digital signatures, certificates, or secure cryptographic verification. Implementing strong authentication and validation methods helps ensure that the data being processed is trustworthy and has not been tampered with.

Business impact : CWE-345 Insufficient Verification of Data Authenticity occurs when a system fails to properly verify whether data originates from a trusted source or has been tampered with during transmission or processing. This weakness can allow attackers to inject malicious or altered data into an application, leading to unauthorized actions or incorrect system behavior. From a business perspective, exploitation of this vulnerability can compromise the integrity of critical business data.

Vulnerability Path : <https://cwe.mitre.org/data/definitions/345.html>



8. Vulnerability name : Improper Output Neutralization for Logs

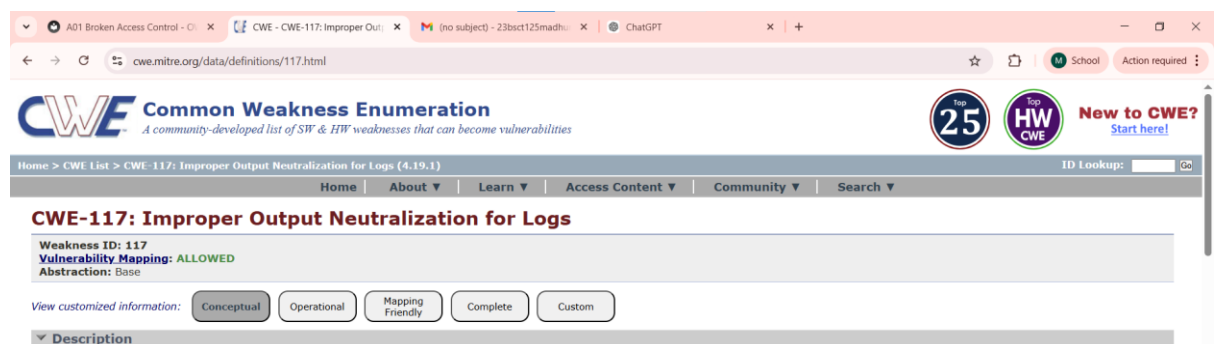
CWE :117

Description : Improper Output Neutralization for Logs is a security vulnerability that occurs when an application writes untrusted input directly into log files without properly sanitizing or neutralizing it. Attackers can exploit this weakness by inserting malicious characters or log entries that may alter the structure of the logs, hide malicious activities, or mislead administrators during log analysis. This can

compromise the reliability and integrity of log records. Properly validating and neutralizing user input before writing it to logs helps maintain accurate and secure logging.

Business Impact : CWE-117 Improper Output Neutralization for Logs occurs when an application fails to properly sanitize or neutralize user-controlled input before writing it to log files. This vulnerability allows attackers to inject malicious content into logs, potentially altering log records or inserting fake log entries. From a business perspective, this can significantly affect the integrity and reliability of logging systems. Attackers may manipulate logs to hide malicious activities, mislead security teams, or create false records that complicate incident investigations.

Vulnerability Path : <https://cwe.mitre.org/data/definitions/117.html>



9.Vulnerability name : Generation of Error Message Containing Sensitive Information
CWE : 209

Description : Generation of Error Message Containing Sensitive Information is a security vulnerability that occurs when an application reveals sensitive information in its error messages. These messages may include details such as system paths, database queries, configuration settings, or internal application logic. Attackers can use this information to better understand the system and exploit other vulnerabilities. To prevent this issue, applications should display generic error messages to users while logging detailed error information securely for administrators.

Business Impact : CWE-209 Generation of Error Message Containing Sensitive Information occurs when an application displays detailed error messages that reveal sensitive system information to users. These messages may expose internal system paths, database queries, configuration details, or software components. From a business

perspective, this vulnerability can significantly increase the risk of targeted cyberattacks. Attackers can use the exposed information to better understand the system's architecture and identify additional vulnerabilities to exploit. This may lead to unauthorized access, data breaches, or further attacks such as injection or system compromise.

Vulnerability Path : <https://cwe.mitre.org/data/definitions/209.html>

STAGE – 2

TITLE: Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

S. No	Vulnerability Name	Severity	Plugins
1.	Nessus SYN scanner	INFO	11219
2.	Hypertext transfer protocol	INFO	24260
3.	HTTP Server type version	INFO	10107
4.	Web Server robots.txt information disclosure	INFO	10302
5.	Service Detection	INFO	22964

1. VULNERABILITY NAME: NESSUS SYN SCANNER

SEVERITY: INFO

PLUGIN: 11219

Description:

The *Nessus SYN Scanner* finding indicates that a system or network device has detected a **TCP SYN scan**, which is a technique used to identify open ports on a target host. In a SYN scan, the scanner sends TCP SYN packets to various ports and analyzes the responses to determine whether the ports are open, closed, or filtered without completing the full TCP handshake. This method is commonly used by vulnerability scanning tools such as Nessus developed by Tenable to discover exposed services during security assessments. While SYN scanning is legitimate in authorized vulnerability assessments, its detection may indicate that a system is being actively scanned for potential entry points.

Business Impact:

If SYN scanning activity is performed by unauthorized sources, it may indicate reconnaissance by attackers attempting to identify open ports and vulnerable services within the organization's network. This information can help attackers plan further exploitation attempts, potentially leading to unauthorized access, service disruption, or data breaches. Continuous scanning may also increase network noise and security alerts, making it harder to distinguish real attacks from normal traffic and increasing the workload for security teams.

Solution:

Organizations should ensure that only authorized vulnerability scanners are allowed to perform scans within the network. Implement firewall and intrusion detection/prevention systems to monitor and restrict unauthorized port scanning activities. Network administrators should review the source of the SYN scan, verify whether it originates from an approved security tool such as Nessus, and block or investigate any suspicious or unknown sources. Additionally, limiting exposed services and applying proper network segmentation can reduce the attack surface and minimize the effectiveness of reconnaissance attempts.

2. VULNERABILITY NAME: HYPERTEXT TRANSFER PROTOCOL

SEVERITY: INFO

PLUGIN: 24260

Description:

The **Hypertext Transfer Protocol (HTTP)** vulnerability refers to the use of the unsecured HTTP protocol for transmitting data between a client and a web server. HTTP sends data in **plain text**, meaning that information such as login credentials, session tokens, and other sensitive data can be intercepted or modified by attackers during transmission. Because the communication is not encrypted, it is vulnerable to attacks such as eavesdropping, session hijacking, and man-in-the-middle attacks. Security best practices from organizations such as OWASP recommend using encrypted communication protocols to protect sensitive information transmitted over networks.

Business Impact:

If a web application uses HTTP instead of secure communication, attackers may intercept sensitive data transmitted between users and the server. This could lead to unauthorized access to user accounts, exposure of confidential business information, or manipulation of transmitted data. Such incidents can result in data breaches, loss of customer trust, reputational damage, and potential regulatory or compliance issues for the organization.

Solution:

To mitigate this vulnerability, organizations should enforce the use of **HTTPS** instead of HTTP for all web communications. Implement an SSL/TLS certificate and configure the web server to automatically redirect all HTTP requests to HTTPS. Additionally, enable strong encryption protocols and disable insecure versions of SSL/TLS to ensure secure communication. Following secure development and configuration practices recommended by organizations like OWASP will help protect sensitive data during transmission.

3. VULNERABILITY NAME: HTTP SERVER TYPE VERSION

SEVERITY: INFO

PLUGIN: 10107

Description:

The **HTTP Server Type and Version Disclosure** vulnerability occurs when a web server reveals detailed information about the server software and its version in HTTP response headers or error pages. For example, the server may expose headers such as `Server: Apache/2.4.49` or `Server: nginx/1.18.0`. This information disclosure allows attackers to identify the exact server software and version being used. If the disclosed version has known vulnerabilities, attackers can use publicly available exploits to target the system. Security guidance from organizations such as OWASP recommends minimizing server information disclosure to reduce the risk of targeted attacks.

Business Impact:

Exposing the HTTP server type and version can assist attackers in reconnaissance activities by providing information about the technologies used in the environment. This makes it easier for attackers to identify known vulnerabilities associated with that specific server version and launch targeted attacks. Successful exploitation could lead to unauthorized access, service disruption, data compromise, or further attacks on internal systems, which may impact business operations and damage the organization's reputation.

Solution:

To mitigate this vulnerability, configure the web server to hide or suppress server version information in HTTP response headers and error messages. For example, modify server configuration settings to disable or limit the `Server` and `X-Powered-By` headers. Additionally, keep web server software such as Apache HTTP Server or Nginx updated with the latest security patches and follow secure configuration guidelines recommended by OWASP. This reduces the risk of attackers identifying and exploiting known vulnerabilities.

4. VULNERABILITY NAME: WEB SERVER ROBOTS.TXT INFORMATION DISCLOSURE

SEVERITY: INFO

PLUGIN: 10302

Description:

The **Web Server robots.txt Information Disclosure** vulnerability occurs when the `robots.txt` file on a web server reveals sensitive or restricted directories that should not be indexed by search engines. The `robots.txt` file is used to instruct search engine crawlers

about which parts of a website should not be crawled or indexed. However, because this file is publicly accessible, attackers can view it and identify hidden or sensitive paths such as administrative panels, backup folders, or internal resources. Security organizations such as OWASP consider this an information disclosure issue because it unintentionally exposes the structure of the web application.

Business Impact:

If sensitive directories or files are listed in the `robots.txt` file, attackers can use this information to locate restricted areas of the application, such as login portals, configuration files, or backup directories. This may assist attackers in conducting further reconnaissance or launching targeted attacks against these resources. In some cases, this could lead to unauthorized access, exposure of confidential data, or exploitation of vulnerable components, which can impact business operations and compromise the security of the application.

Solution:

To mitigate this vulnerability, avoid listing sensitive or confidential directories in the `robots.txt` file. Instead of relying on `robots.txt` to restrict access, implement proper access control mechanisms such as authentication and authorization to protect sensitive resources. Additionally, review the web application structure and ensure that administrative or internal directories are not publicly accessible. Following secure configuration practices recommended by organizations such as OWASP can help reduce the risk of information disclosure.

5. VULNERABILITY NAME: SERVICE DETECTION

SEVERITY: INFO

PLUGIN: 22964

Description:

The **Service Detection** vulnerability indicates that a scanning tool has successfully identified services running on open ports of a host. During this process, the scanner interacts with the target system to determine the type of service (such as web server, database, or mail server) and sometimes the version of the software running on those ports. Tools such as Nmap and vulnerability scanners like Nessus commonly perform service detection during security assessments. While this activity itself is not a direct vulnerability, it reveals information about the services available on the system, which can assist attackers in identifying potential targets.

Business Impact:

If attackers are able to detect and identify running services on a system, they may use this information to perform further reconnaissance and search for known vulnerabilities associated with those services. This could enable targeted exploitation attempts, unauthorized access, or disruption of services. In environments where sensitive systems are exposed, service detection may increase the risk of attacks against critical infrastructure, potentially affecting system availability, data confidentiality, and overall business operations.

Solution:

To mitigate risks associated with service detection, organizations should minimize the exposure of unnecessary services by disabling unused ports and services on servers and network devices. Implement firewall rules and network segmentation to restrict access to critical services only from trusted sources. Additionally, regularly update and patch services to address known vulnerabilities and follow security best practices recommended by organizations such as OWASP. Monitoring network traffic and intrusion detection systems can also help identify and respond to unauthorized scanning activities.

Stage – 3

Title: "ZPA enforces a Zero Trust architecture by dynamically granting users the minimum necessary access to specific applications based on verified identity, rather than providing broad network entry."

Zero Trust Network Access (ZTNA): ZTNA is a cybersecurity framework that replaces traditional, broad network access with an identity-centric "never trust, always verify" model, ensuring that no user or device is granted entry by default. By establishing secure, application-specific tunnels rather than open network connections, ZTNA keeps internal resources invisible to the public internet and restricts users to only the specific tools they need. This approach effectively shrinks the attack surface and prevents lateral movement, securing the environment regardless of whether a user is connecting from inside or outside the corporate perimeter.

Zscaler Private Access (ZPA): Zscaler Private Access (ZPA) replaces traditional network-level entry with a service-initiated model that brokers secure connections between users and specific private applications via the Zscaler cloud. By utilizing ZPA App Connectors to facilitate these connections, the platform ensures users are never actually placed on the corporate network, keeping internal IP addresses and infrastructure completely invisible to the public. Access is governed by dynamic, context-aware policies—incorporating identity, device posture, and location—to ensure that users can only see and reach the specific applications they are explicitly authorized to use, effectively eliminating the risk of lateral movement.

Principle of Least Privilege: The Principle of Least Privilege (PoLP) is a foundational security strategy that restricts users, devices, or processes to the absolute minimum access required for their specific functions, shifting from broad "fat" network segments to "thin," application-specific permissions. By enforcing this via Zscaler Private Access (ZPA), organizations ensure that internal infrastructure remains invisible and inaccessible, preventing attackers from pivoting to other systems even if credentials are compromised. This granular control is essential for reducing the internal attack surface, mitigating insider threats, and strictly limiting the "blast radius" of any potential security incident.

Identity-Aware Access: Identity-aware access shifts security from location-based trust to a model where permissions are strictly tied to a user's verified identity and real-time context. By integrating directly with Identity Providers (IdPs) like Okta, Azure AD, or Ping Identity, Zscaler Private Access (ZPA) leverages existing user attributes and roles to build granular policies. For instance, a rule might only permit a "Finance" team member to access "SAP" if they are using a company-managed device during business hours. This ensures that access is tightly coupled with who the user is and the health of their device, rather than relying on easily spoofed or static IP addresses.

Application-Specific Access Policies: Application-specific access policies serve as ZPA's core enforcement mechanism, replacing broad subnet rules with precise, plain-language logic targeted at individual applications. These policies allow for deep micro-segmentation, where internal systems like HR portals, development servers, and databases are isolated behind their

own unique sets of conditions. By incorporating variables such as identity, location, and device posture, ZPA ensures that being authenticated for one specific tool provides no visibility or connectivity to any other. This "dark" application environment effectively prevents unauthorized users from even seeing resources they aren't explicitly permitted to use.

1. Zscaler Cloud (Service Edge): The Zscaler Cloud, or Service Edge, is a globally distributed, multi-tenant platform that serves as the intelligent broker for all user-to-application connections. Rather than sitting directly in the data path, it handles critical control-plane functions such as user authentication, dynamic policy enforcement, and connection brokering. By decentralizing these operations across a worldwide fabric, ZPA ensures that security decisions are made at the edge, closest to the user, providing low-latency access while maintaining a single, unified point of management.

2. App Connectors: App Connectors are lightweight software components, deployed as VMs or containers, that reside within your private data center or public cloud environments (like AWS or Azure). Their primary role is to establish outbound-only "inside-out" tunnels to the Zscaler Cloud, which completely eliminates the need for open inbound firewall ports or public-facing IP addresses. When a user is authorized, the App Connector receives the request from the Zscaler broker and forwards the traffic to the internal application, ensuring the private infrastructure remains invisible to the internet.

3. ZPA Client Connector: The ZPA Client Connector is a unified agent installed on end-user devices across Windows, macOS, Linux, iOS, and Android to ensure a seamless "office-like" experience. It is responsible for authenticating the user's identity, verifying the device's security posture (such as disk encryption or antivirus status), and establishing a secure micro-tunnel to the Zscaler Service Edge. By constantly monitoring the device's health and location, the Client Connector ensures that application access is always contextual and revoked immediately if a device becomes non-compliant.

4. Application Segments: Application Segments are logical groupings within the ZPA administrative console that define exactly which private resources—such as web apps, SSH servers, or databases—are available for access. Rather than defining access by IP addresses or subnets, administrators use these segments to organize resources by function or department. This abstraction allows for precise micro-segmentation, ensuring that once an Application Segment is assigned to a policy, only authorized users can see or interact with those specific resources, while all other segments remain "dark."

Implementation Methodology: The deployment of ZPA follows a structured, phased approach to ensure minimal disruption and maximum security efficacy:

Phase 1: Discovery and Inventory: The Discovery and Inventory phase focuses on mapping the existing digital landscape by identifying every internal application, then categorizing them by department, user base, and criticality. Leveraging tools like network scanners, CMDBs, and stakeholder interviews, security teams build a comprehensive master list of resources. This groundwork is essential for moving away from broad network access toward a structured, application-centric model where every asset is accounted for before migration begins.

Phase 2: Connector Deployment: During Connector Deployment, ZPA App Connectors are provisioned as lightweight virtual machines or containers within the specific data centers or cloud environments where applications reside. These connectors are configured to initiate outbound-only connections to the Zscaler Cloud, effectively "dialing out" to establish a secure path. This critical step ensures that internal resources can be reached by authorized users without requiring any holes to be poked in the corporate firewall.

Phase 3: Application Onboarding: Application Onboarding involves defining specific Application Segments within the ZPA administrative console to represent private resources. Administrators map the internal IP addresses and port requirements for each service and associate them with the relevant App Connector groups. This process transforms raw network assets into logical, reachable entities within the Zscaler fabric, setting the stage for granular, identity-based access control.

Phase 4: Policy Creation: The Policy Creation phase centers on integrating ZPA with the organization's Identity Provider (IdP), such as Azure AD or Okta, to enable context-aware security. Security teams draft granular rules that dictate access based on user group membership, device health, and geographic location. These policies are typically tested in a simulation or "Log Only" mode first, allowing administrators to verify that the rules work as intended without accidentally blocking legitimate business traffic.

Phase 5: Pilot and Rollout: In the Pilot and Rollout phase, a controlled group of users is migrated to ZPA to validate the user experience and connectivity in a real-world production environment. This stage allows for the collection of feedback, performance monitoring, and policy refinement based on actual usage patterns. Once the pilot is successful, the organization gradually transitions the broader user population away from legacy access methods, ensuring a smooth and stable migration.

Phase 6: VPN Decommissioning: The final stage, VPN Decommissioning, occurs once all users and applications have been successfully migrated to the ZTNA framework. By shutting down the legacy VPN infrastructure, the organization officially eliminates a massive, public-facing attack surface and removes the risks associated with broad network-level entry. This marks the completion of the transition to a true Zero Trust environment, where the internal network is fully "dark" to the outside world.

Tools and Technologies: The ZPA ecosystem relies on a stack of integrated technologies to deliver a seamless Zero Trust experience, centered around Zscaler Private Access as the primary broker. This architecture is powered by Identity Providers (IdPs) like Azure AD or Okta for authentication, and Endpoint Protection Platforms (EPP) such as CrowdStrike or Microsoft Defender to verify device hygiene before granting access. On the infrastructure side, Virtualization Platforms and Public Clouds host the App Connectors, often deployed at scale using Infrastructure as Code (IaC) tools like Terraform. Finally, SIEM platforms provide centralized logging for security correlation, while Digital Experience Monitoring (DEM) tools ensure optimal connection quality and application performance for the end user.

Integration with Identity Providers: Integrating ZPA with an Identity Provider (IdP) is the critical bridge that enables identity-aware security, typically established via SAML or LDAP protocols. This connection allows ZPA to synchronize user attributes and group memberships, ensuring that access policies can be written using familiar business roles rather than technical network tags. When a user attempts to connect through the Zscaler Client

Connector, they are redirected to the corporate IdP for credential verification and Multi-Factor Authentication (MFA). Only after a successful handshake does ZPA receive the necessary identity metadata to evaluate policies and dynamically "unhide" the specific applications the user is authorized to reach.

Device Posture Checking: Device posture checking allows ZPA to move beyond simple identity verification by assessing the real-time security health of a connecting endpoint. Through integrations with EDR and MDM tools like CrowdStrike or Microsoft Intune, ZPA evaluates critical safety markers such as antivirus status, disk encryption, OS patch levels, and the absence of jailbreaking on mobile devices. If a device's risk score spikes or it fails a compliance check—for example, if a firewall is disabled—ZPA can automatically block access or trigger a remediation workflow, ensuring that compromised or "unhealthy" devices never touch sensitive data.

Micro-tunneling and Application Visibility: Micro-tunneling is the architectural backbone of ZPA, establishing secure, isolated TLS/DTLS tunnels that link a user directly to a specific application rather than a network segment. By proxying traffic at Layer 7, ZPA ensures the user's device is never actually placed on the corporate network, making internal infrastructure completely invisible to the user. This "dark" network approach prevents unauthorized users from performing port scans or vulnerability reconnaissance, as they can only resolve DNS names for the specific resources they are explicitly authorized to use.

Incident Response in a ZPA Environment: In a ZPA-protected environment, incident response becomes faster and more precise by enabling security teams to instantly isolate compromised accounts or devices with a single click. Because users lack network-level access, the blast radius of a breach is naturally contained, preventing an attacker from lateral movement or scanning the internal environment. Furthermore, ZPA provides a granular "paper trail" for forensics—detailing exactly which apps were accessed and for how long—while allowing administrators to block high-risk services without disrupting the user's access to other essential business tools.

Understanding ZPA: Key Takeaways: Zscaler Private Access marks a revolutionary shift to cloud-native, identity-centric security that removes the need for hardware-heavy VPNs. By keeping internal infrastructure invisible to the internet, ZPA effectively shields the organization from reconnaissance and DDoS attacks while scaling effortlessly to support thousands of users across hybrid and multi-cloud environments. Ultimately, this model enhances the user experience by providing direct, low-latency connections to applications, eliminating the performance bottlenecks typically associated with backhauling traffic through traditional data centers.

Stage 1: Web Application Testing Web application testing is a foundational process that ensures the security, functionality, and reliability of digital software by evaluating user interfaces and performance before deployment. By identifying and rectifying vulnerabilities early, this practice enhances application quality and bolsters resilience against evolving cyber threats, ultimately delivering a seamless and secure experience in an increasingly online world.

Stage 2: Nessus Security Insights The Nessus vulnerability report provides a critical security baseline by identifying and categorizing risks to ensure high-priority threats receive immediate remediation through patching and reconfiguration. By aligning findings with

industry standards like OWASP, the report emphasizes the necessity of continuous monitoring and sustained vigilance to maintain a robust security posture against a dynamic threat landscape.

Stage 3: Zero Trust Transformation The implementation of Zscaler Private Access (ZPA) marks a strategic shift from vulnerable, network-centric security to a resilient Zero Trust architecture that enforces least-privilege access. This project successfully renders private applications invisible to the internet, neutralizing lateral movement and providing a high-performance, future-ready foundation for secure digital transformation across a distributed workforce.

Future Scope:

Stage 1: Evolving Demand and Complexity The future of web application testing will be driven by surging demand for secure online services and the integration of advanced technologies like AI, IoT, and blockchain. As applications become more complex, testers will shift focus toward specialized security compliance (such as GDPR), cross-browser responsiveness for mobile users, and rigorous performance testing to handle massive traffic spikes.

Stage 2: AI and Infrastructure Security Vulnerability discovery is set to be revolutionized by AI-driven testing tools capable of scanning vast datasets to identify flaws that elude human detection. The scope will also expand to secure emerging architectures—including serverless functions, containerized environments like Kubernetes, and blockchain-based assets—ensuring that every layer of the modern digital stack is hardened against compromise.

Stage 3: Mature Zero Trust & SASE Moving forward, the organization can evolve its security by converging ZPA with Zscaler Internet Access (ZIA) into a unified SASE platform, creating a single policy framework for all traffic. Future initiatives will include extending access to third-party vendors, implementing AI-driven anomaly detection to identify suspicious behavior in real-time, and securing "east-west" traffic between cloud workloads and IoT/OT devices through automated micro-segmentation.

Topics Explored:

1. Zscaler
2. Zero trust
3. ZPA
4. VPN
5. MISP
6. College Network Information
7. Deployment of SOC in the College
8. Threat Intelligence
9. Incident Response

Tools Explored:

- Nessus Scanner

